# RabbitMQ学习

**RabbitMQ** 是一种 **消息队列（Message Queue）中间件**

> 用来让不同程序之间 **通过“消息”进行通信和解耦** 的软件。

它在 **分布式系统、微服务架构、后台任务处理** 中非常常见。

**消息队列** 是一种 **程序之间传递消息的通信机制。**
它的核心思想非常简单：

> **一个程序把消息放进队列，另一个程序从队列里取消息处理。**

就像 **排队取餐** 一样。

---

## 举个场景来理解

假设你开发了一个 **电商网站**，用户下单后需要做很多事情：

1. 扣库存
2. 发短信通知
3. 生成订单
4. 记录日志
5. 推荐商品

如果全部 **同步执行**：

```
用户下单
    ↓
扣库存
    ↓
发短信
    ↓
记录日志
    ↓
推荐系统
    ↓
返回成功
```

问题：

- 用户**要等很久**
- **一个服务挂了整个系统卡住**
- **服务之间耦合严重**

**“服务之间耦合严重”** 是软件架构里非常常见的一个概念。
简单理解就是：

> **一个服务强依赖另一个服务，彼此绑得很死，改一个地方就会影响很多系统。**

这种情况就叫 **高耦合（High Coupling）**。

---

**使用 RabbitMQ 后**

```
用户下单
   ↓
发送一个消息到 RabbitMQ
   ↓
RabbitMQ存消息
   ↓
各个系统自己去拿消息处理
```

变成：

```
订单系统  → RabbitMQ → 库存系统
                     → 短信系统
                     → 推荐系统
                     → 日志系统
```

这样：

- 用户下单 **马上返回**
- 后台慢慢处理
- 系统之间 **解耦**

---

# RabbitMQ 的核心作用

主要解决 **3个问题**：

## 异步处理

```
用户注册
→ 发送邮件
→ 发送短信
→ 推荐好友
```

这些都可以 **异步执行。**

---

## 系统解耦

没有 RabbitMQ：耦合严重。

```
A系统 → 调 B系统 → 调 C系统
```

有 RabbitMQ：系统之间 **不直接依赖**。

```
A系统 → RabbitMQ → B系统
                 → C系统
```

---

## 削峰填谷（抗高并发）

例如秒杀：数据库扛不住。

```
10000人同时下单
```

RabbitMQ可以：避免服务器崩溃。

```
请求 → RabbitMQ排队
        ↓
    慢慢处理
```

---

# RabbitMQ的基本结构

```
生产者 (Producer)
        ↓
     Exchange
        ↓
      Queue
        ↓
消费者 (Consumer)
```

流程：

```
程序A 发送消息
   ↓
Exchange交换机
   ↓
Queue队列
   ↓
程序B消费消息
```

简单图：

```
Producer
    ↓
 Exchange
    ↓
  Queue
    ↓
 Consumer
```

---

# RabbitMQ常见应用场景

## 微服务通信

SpringBoot 微服务：

```
订单服务 → RabbitMQ → 库存服务
```

---

## 订单系统

```
用户下单
   ↓
RabbitMQ
   ↓
库存扣减
   ↓
物流系统
```

---

## 日志系统

```
应用 → RabbitMQ → 日志服务器
```

---

## 异步任务

```
用户上传图片
```

可以：

```
上传 → RabbitMQ → 图片压缩服务
                → 图片识别服务
```

---

# RabbitMQ vs Redis

| 对比 | RabbitMQ | Redis |
| --- | --- | --- |
| 类型 | 专业消息队列 | 缓存数据库 |
| 功能 | 消息传递 | 数据存储 |
| 消息可靠性 | 很强 | 较弱 |
| 复杂路由 | 支持 | 不支持 |

---

# RabbitMQ在Java里的使用

做 **SpringBoot项目** 时非常常见

SpringBoot整合：

```
Spring Boot
Spring AMQP
RabbitMQ
```

发送消息：

```
rabbitTemplate.convertAndSend("queueName","hello");
```

接收消息：

```
@RabbitListener(queues = "queueName")
public void receive(String msg){
    System.out.println(msg);
}
```

---

# 现在学的技术的关系

最近在学：

- Linux
- MySQL
- Redis
- Nacos
- SpringBoot

RabbitMQ 是 **Java后端开发必学组件之一**

完整技术栈通常是：

```
SpringBoot
MySQL
Redis
RabbitMQ
Nacos
Docker
Linux
```

---

**一句话总结**

RabbitMQ就是：

> 一个专门负责 **传递消息、解耦系统、处理高并发任务** 的消息队列服务器。

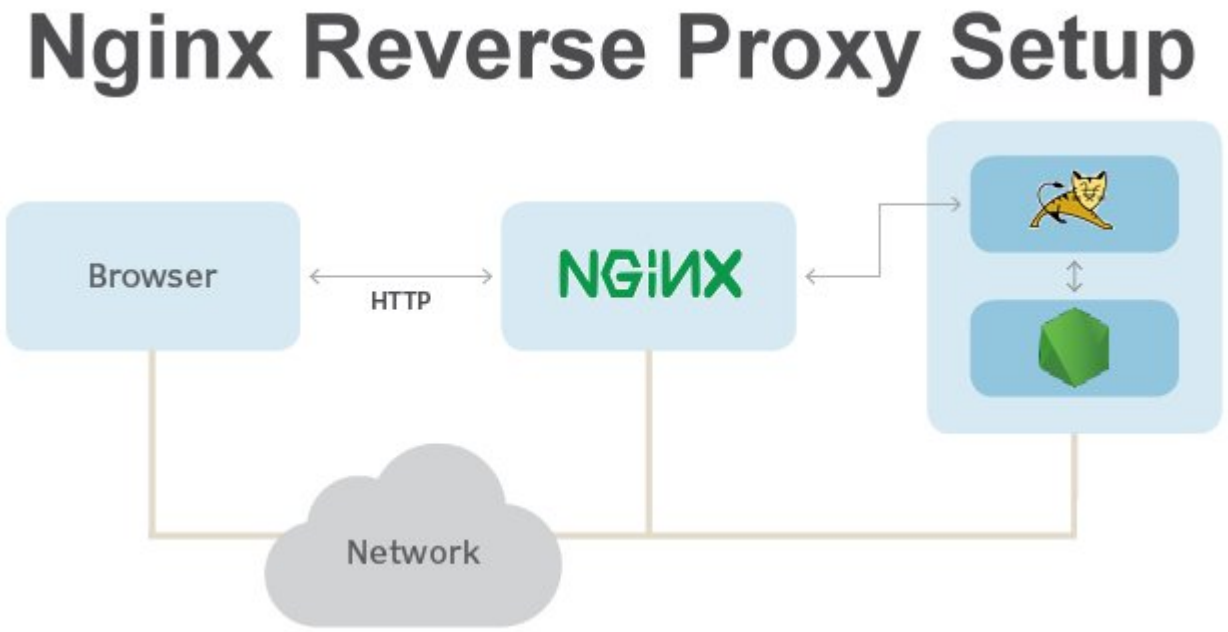
# 有nginx为什么还要RabbitMQ

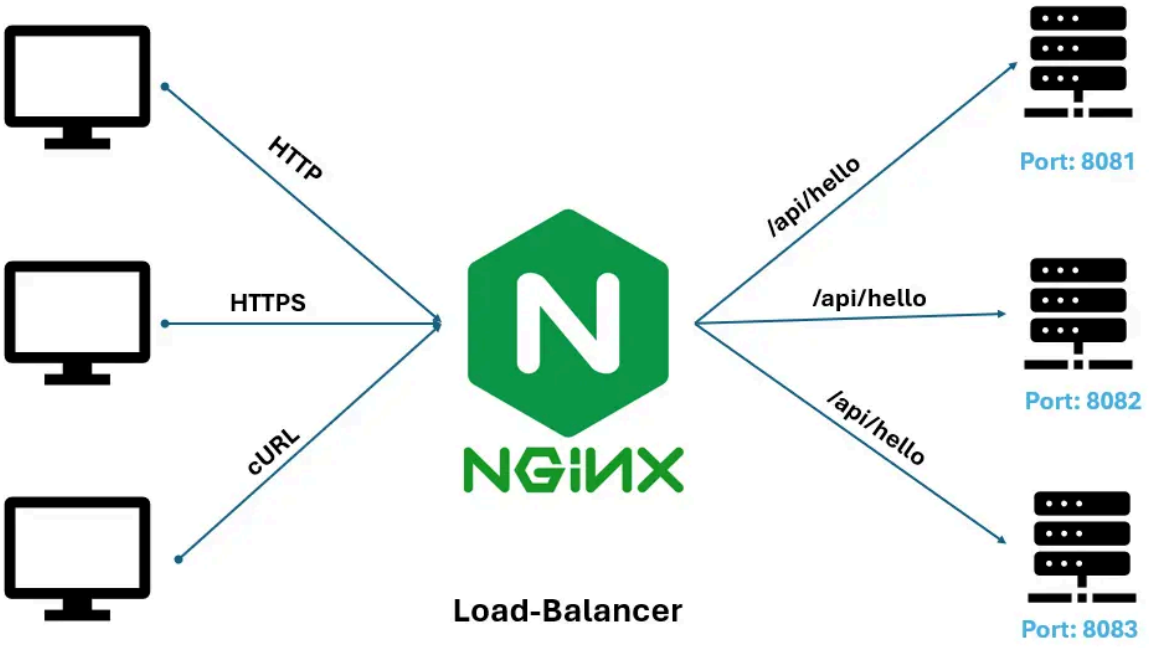
> **Nginx解决的是“请求并发”问题，RabbitMQ解决的是“任务处理”问题。**

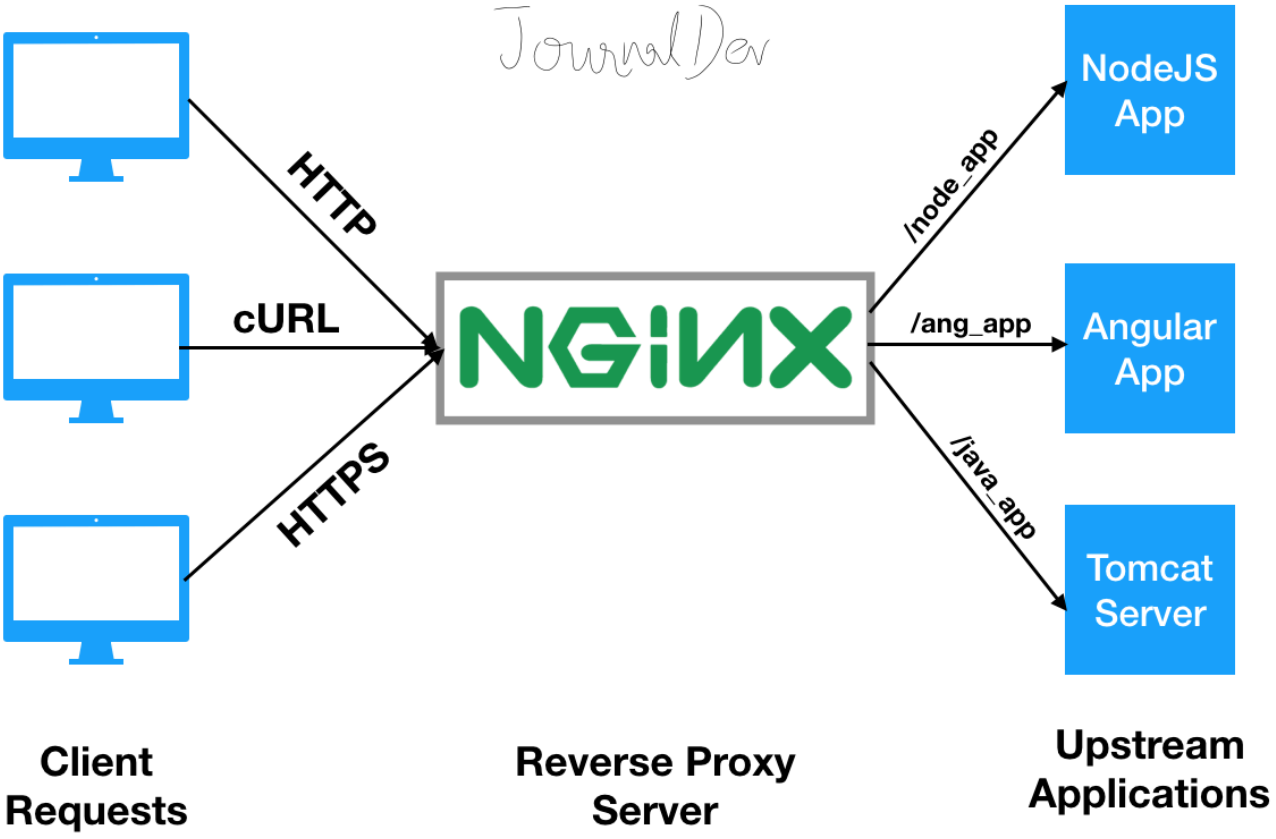
两者在架构里的位置完全不同。

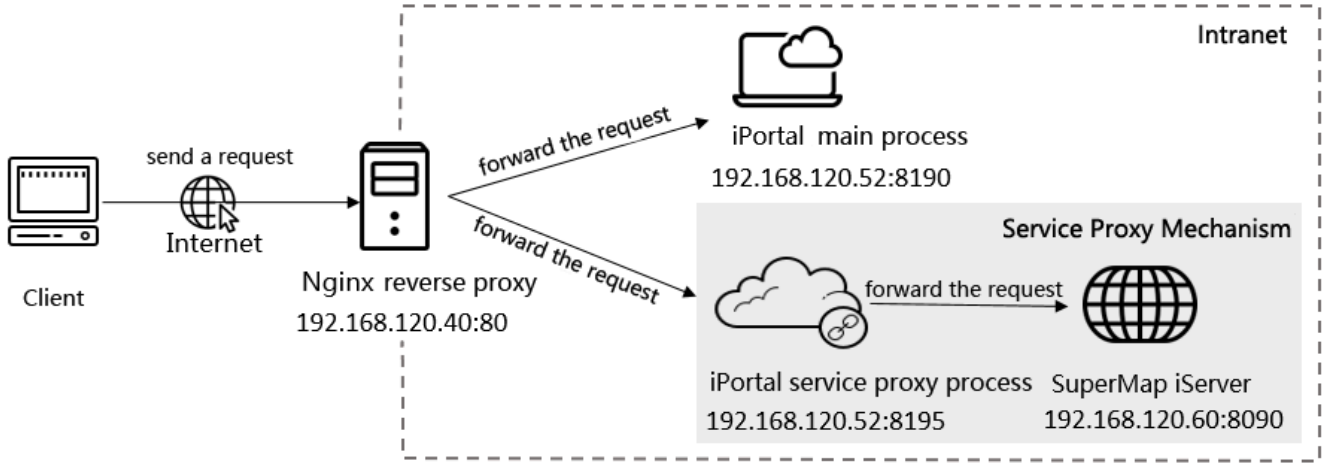
---

# Nginx 是干什么的




HTTP
HTTPS
cURL
NGINX
Load-Balancer
/api/hello
/api/hello
/api/hello
Port: 8081
Port: 8082
Port: 8083


JournalDev
HTTP
cURL
HTTPS
NGINX
/node_app
/ang_app
/java_app
NodeJS App
Angular App
Tomcat Server
Client Requests
Reverse Proxy Server
Upstream Applications




Nginx 本质是一个：

- **Web服务器**
- **反向代理**
- **负载均衡器**

它主要负责：

1. 接收用户请求
2. 转发请求
3. 分配服务器压力

例如：

```
用户请求
   ↓
 Nginx
   ↓
服务器1
服务器2
服务器3
```

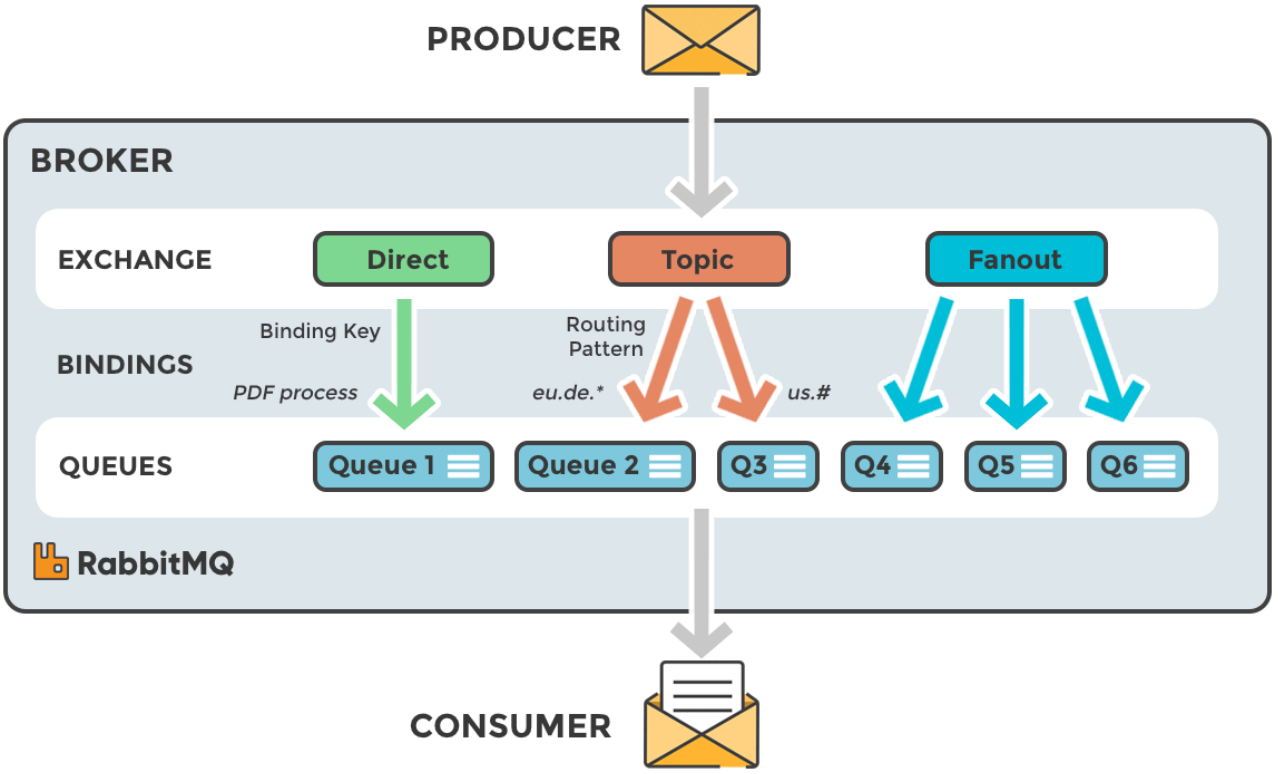
作用：

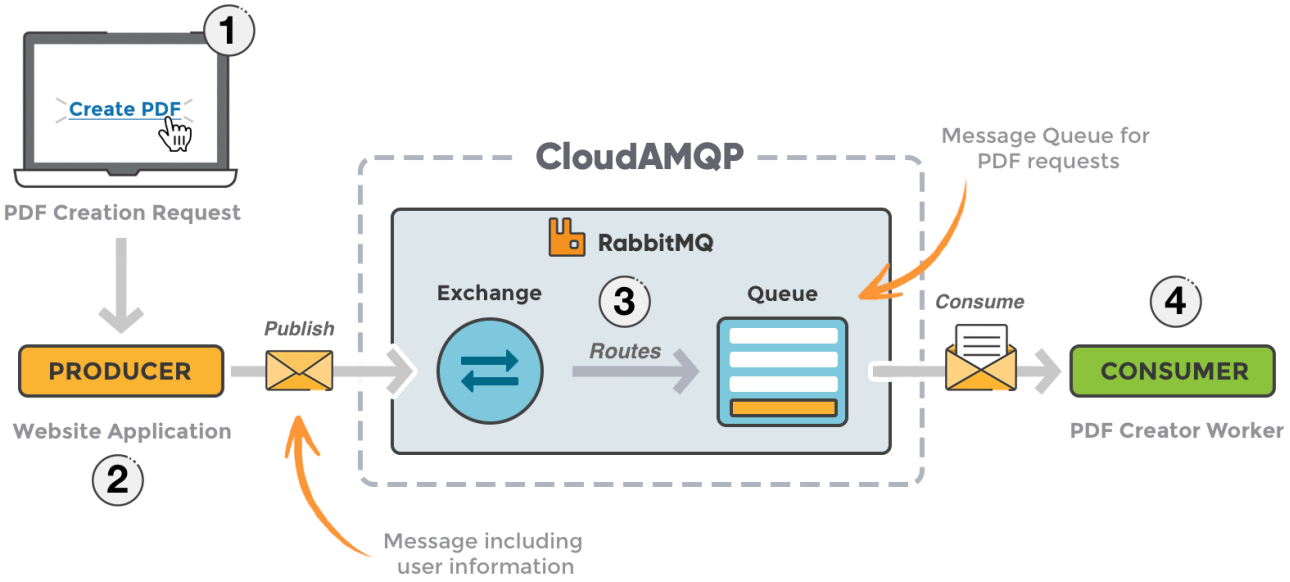
- 分散流量
- 防止服务器被压垮

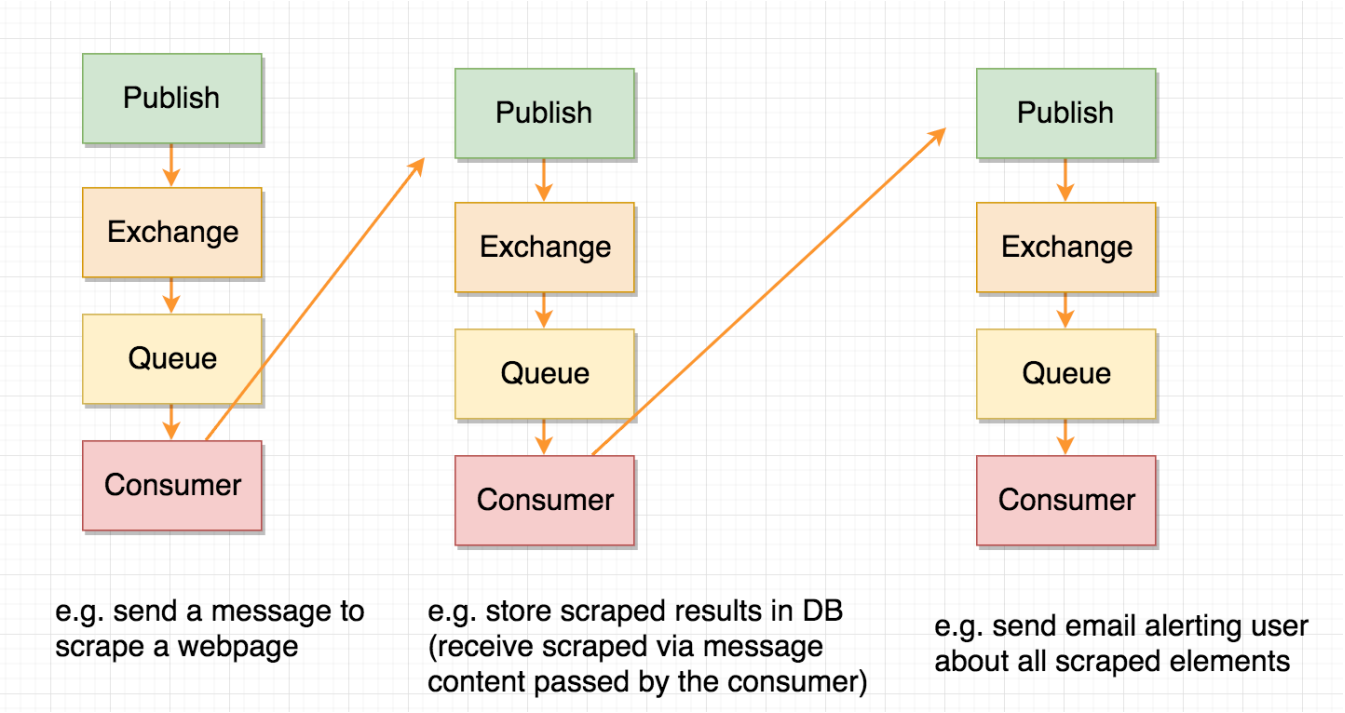
这叫 **负载均衡。**

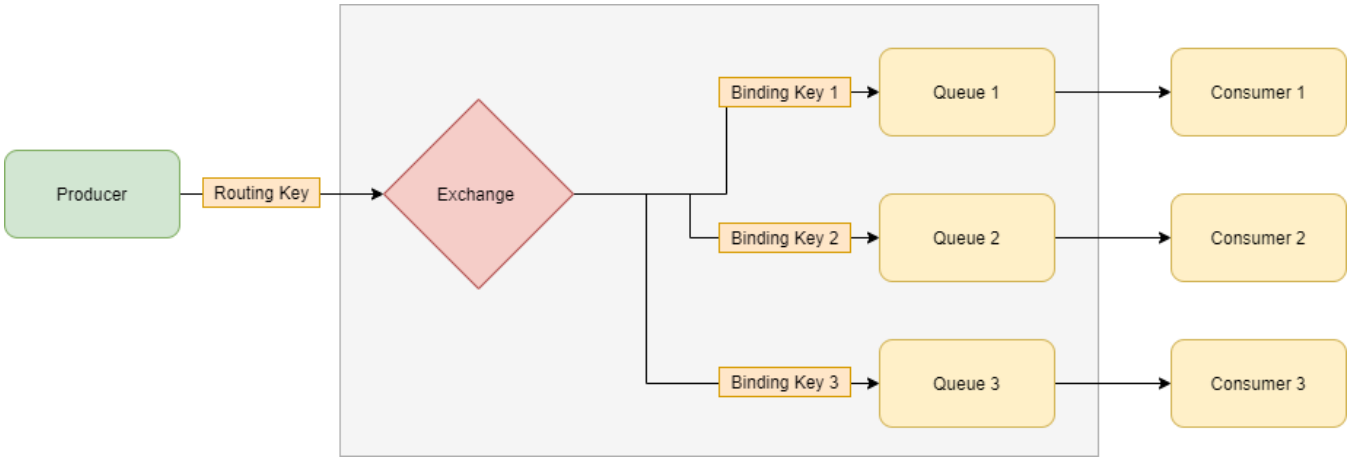
---

# RabbitMQ 是干什么的






RabbitMQ 是:

> **任务排队系统**

用户注册后需要:

```
发送邮件
发送短信
写日志
推荐好友
```

如果同步执行:用户要等很久。

```
用户注册
 ↓
发送邮件
 ↓
发送短信
 ↓
写日志
```

RabbitMQ 的做法：后台慢慢处理。

```
用户注册
 ↓
RabbitMQ
 ↓
邮件服务
短信服务
日志服务
```

---

# 最核心区别（非常重要）

| 对比 | Nginx | RabbitMQ |
| --- | --- | --- |
| 类型 | Web服务器 | 消息队列 |
| 处理对象 | HTTP请求 | 程序任务 |
| 作用 | 分流流量 | 排队任务 |
| 位置 | 系统入口 | 系统内部 |
| 解决问题 | 高并发访问 | 异步任务 |

---

# 真实互联网系统架构

现实系统通常是：

```
用户
 ↓
Nginx
 ↓
应用服务器
 ↓
RabbitMQ
 ↓
后台任务
```

流程：

```
用户下单
 ↓
Nginx
 ↓
订单服务
 ↓
RabbitMQ
 ↓
库存服务
 ↓
短信服务
 ↓
物流服务
```

所以：

- **Nginx 在最外面**
- **RabbitMQ 在系统内部**

---

# 一个特别形象的比喻

想象一个餐厅：

## Nginx

像 **门口的服务员**

负责：

- 接待顾客
- 分配桌子

```
顾客 → 服务员 → 不同桌子
```

解决 **人太多的问题。**

---

## RabbitMQ

像 **厨房里的订单队列**

```
服务员 → 厨房订单 → 厨师做菜
```

厨师慢慢做。

解决 **任务太多的问题。**

---

# 现在学习的技术栈里

你现在接触的架构其实会变成：

```
用户
 ↓
Nginx
 ↓
SpringBoot
 ↓
Redis
 ↓
MySQL
 ↓
RabbitMQ
```

每个组件职责不同：

| 技术 | 作用 |
| --- | --- |
| Nginx | 处理请求 |
| SpringBoot | 业务逻辑 |
| Redis | 缓存 |
| MySQL | 数据存储 |
| RabbitMQ | 异步任务 |

---

💡 **一句话总结**

- **Nginx：解决“访问压力”**
- **RabbitMQ：解决“任务压力”**

---

如果你愿意，我可以再给你讲一个 **很多后端工程师真正理解MQ的关键例子**：

**“为什么秒杀系统必须用 RabbitMQ，而不是 Nginx + 数据库？”**

这个例子非常经典，一看就彻底懂 MQ 的价值。